Approval-Based Elections in the 1D Euclidean Domain

Jan Sznajd

Supervisor: dr hab. inż. Piotr Faliszewski

Institute of Computer Science
AGH University of Science and Technology

This dissertation is submitted for the degree of
Master of Science

May 2021

TODO

0.1 Resources

https://github.com/jasieksz/master-thesis-code/blob/master/flat-code/ch8_main.py

0.2 My Questions

1. Fitzsimmons Lackner 2019, COP - SAT Solver

0.3 10/12/2020

1. Refactor code - vcr ilp detection, profile generation
2. Generate and analyze profiles: 5x5, 4x6, 6x4, random NxM

0.4 17/12/2020

1. Implement ilp MAV
2. Implement VR/CR MAV **?**
3. Compare ilp and poly MAV

0.5 07/01/2021

1. VR/CR MAV **?**

0.6 14/01/2021

1. VR/CR MAV **?**

0.7 11/02/2021

1. write thesis (+- 3 pages)
2. email about CR MAV to Liu/Guo
3. fix CR MAV algo
4. try to proof CR MAV
5. single thread experiments

0.8 18/02/2021

1. investigate CR MAV
2. VCR MAV ideas

3. email about CR MAV to Liu/Guo
4. try to proof CR MAV
5. write thesis
6. 20x20 VCR profiles - random distribution analysis
7. delete some random Candidates for VCR profile to become VR / CR

0.9 25/02/2021

1. investigate CR MAV
2. VCR MAV ideas
3. write thesis

0.10 04/03/2021

1. Chińczycy - wyslac kontrprzykład
2. VCR MAV ideas
3. write thesis - ch.3 (1D Euclidean Pref.) and ch.5 (domain analysis)
4. 20x20 VCR profiles - random distribution analysis
5. delete some random Candidates for VCR profile to become VR / CR
6. Control problem for VCR
7. Justified Representation (Proportional / Extended) for VCR

0.11 11/03/2021

1. VCR MAV ideas
2. write thesis - ch.6 MAV

0.12 18/03/2021

1. research C1P - NP proofs
2. PoC for deleting C's and/or V's for VCR -> CR / VR
3. research Control Problem
4. fix ch.4 and 5, send for the review (monday)

0.13 08/04/2021

1. bigO: *av-cc-dv* naive greedy
2. ch4 (detection) full draft

0.14 15/04/2021

1. C1S NP proofs, (a,b)-matrix, is every matrix from the proof VCR ? or find counter example.
2. VCR algo for AV-CC-DV
3. prepare code for the random distribution generation
4. rewrite ch.3 1D Euclidean Domain

Abstract

(WIP)

I study approval-based elections, where preferences are in the 1D-Euclidean domain. I explore multiple areas, notably:

1. I create domain detection algorithms and evaluate the results.
2. I look for the efficient method of computing election results.

For the first issue I find ???. For the second one I create polynomial complexity algorithm (not yet) and compare it with LP solutions. (WIP)

Contents

TODO **ii**
0.1 Resources ii
0.2 My Questions ii
0.3 10/12/2020 iii
0.4 17/12/2020 iii
0.5 07/01/2021 iii
0.6 14/01/2021 iii
0.7 11/02/2021 iii
0.8 18/02/2021 iii
0.9 25/02/2021 iv
0.10 04/03/2021 iv
0.11 11/03/2021 iv
0.12 18/03/2021 v
0.13 08/04/2021 v
0.14 15/04/2021 v

1 Introduction **1**

2 Preliminaries **2**

3 One Dimensional Euclidean Domain **4**
3.1 Preference Restrictions 4
3.2 Euclidean Preference Restrictions 4

4 Voter Candidate Range Profile Detection **6**
4.1 Profile Structure Detection 6
4.2 Integer Linear Program 7
4.3 (WIP) My VCR Detection Algorithm 9
4.4 (WIP) C1P Detection - ILP vs PQTree Performance Comparison 9

5 Voter Candidate Range Profiles **10**
5.1 Domain analysis 10
5.2 (WIP) How many VCR profiles are not VR nor CR? 10

6 HD-Minimax Scoring Rule Algorithms **14**
6.1 Hamming Distance Minimax Approval Voting 14
6.2 (WIP) My algorithm for VCR profiles 15

7 (WIP) Approval deletion for VCR profile to become VR or CR **16**

8 Control Problem **17**
8.1 Overview 17
8.1.1 AV-CC-DV for Single Peaked (VR = CI = row C1P = $r_c = 0$) . . . 17
8.1.2 AV-CC-DV for Candidate Range = VI = column C1P = $r_v = 0$ . . . 18
8.2 AV-CC-DV for the VCR profiles 18
8.2.1 Sketch Proof - CR case 19
8.2.2 Sketch Proof - VR case 20
8.2.3 Sketch Proof - VCR (not C1P) case 22

9 Evaluation **24**
9.1 Conclusions 24
9.2 Future Work 24

Bibliography **25**

Chapter 1

Introduction

Elections are ubiquitous element of our society in one form or another. For example choosing members of parliament or a winner of a competition. In a day-to-day activities we need to decide on a meeting time or select recommended movies in a streaming service. Each of these choices may require different type of election and scoring rule, based on the underlying properties of the choice (i.e. voters should be represented proportionally by members of a government)

Reseach objectives

In this work I am going to focus on the *Approval Elections*, that is where each voter provides a subset of candidates that he approves of and disapproves the rest. In general many social choice problems are known to be computationally hard. However they become tractable under some restrictions on the structure of the preferences. Elkind and Lackner (2015) explored well known restrictions in the setting of approval preferences, introduced several new one-dimensional domain restrictions such as Dichotomous Euclidean (sprawdzic kiedy/kto wprowadził pojęcie - spatial pref, multidim unfolding) and derivatives (czy to dobre okreslenie na VR CR?). Faliszewski (2021) generalized this model for the N-dimensional euclidean space calling it Voter Candidate Range and explored two-dimensional case.

The goal of this work is to explore properties of the one-dimensional model. Search for the profile detection polynomial complexity algorithm. I will try as well to extend solution of **?** for computing MAV score with polynomial complexity to the VCR model.

Contributions

I provide linear programming preference detection algorithm and ...

Chapter 2

Preliminaries

In this chapter, we introduce the definitions and notations used in this work.

Approval Elections

Election is a pair $E = (C, V)$, where $C = \{c_1, ..., c_m\}$ is a set of candidates and $V = \{v_1, ..., v_n\}$ is a set of voters. Each voter $v \in V$ approves a subset of the candidates and disapproves the remaining candidates. I will be referring to it as a *approval set* and for each voter $v \in V$ denote it as $A(v)$, when using A just by itself I refer to all approval sets in the given election (preferences).

Committee Selection Rules

Voting rules are algorithms used to select winners (committee) of a given election. Each rule has different properties and must be selected accordingly to the goal of the election. For the detailed overview of multiwinner voting rules I point to Faliszewski et al. (2017).

WIP: Jakie jeszcze tematy/pojęcia tu opisać

Integer Linear Program

Integer linear programming (ILP) is a common approach to solve NP-hard problems, furthermore it can be used effectively for other problems as well. (WIP: czy zdanie od furthermore ... jest potrzebne) To define a computational problem as an integer linear program, we have to create a system of linear inequalities (constraints) (WIP: Sprawdzic czy na pewno inequalities == constraints) and optionally a linear function to optimize, alternatively we can just check feasibility of the constraints. Next to obtain a solution we have to use an optimization solver. Popular choices are Gurobi or CPLEX which are state-of-the-art general purpose solvers.

(WIP: Czy przenieść do Preliminaries + opisać rodzaje zmiennych w ILP, etc. ?)

Chapter 3

One Dimensional Euclidean Domain

3.1 Preference Restrictions

(WIP: Motywacja, po co restrykcje, czy moge powtorzyc Introduction?)

Many social choice problems such as winner determination, or control problem are known to be computationally hard. To deal with it we can use approximate or parametrized algorithms (WIP: Podać przykłady metod / cytacje). The alternative approach is to introduce restrictions on the preference domains. (WIP: Cytacje research'u) In the last few years this area of research advanced a lot, providing multiple efficient algorithms for the popular committee selection rules, and other problems.

3.2 Euclidean Preference Restrictions

Voter Candidate Range

Given an election $E = (C, V)$, we say that the voters have *1D-Euclidean* preferences if for every agent (voter or candidate) there exists a point x_a and nonnegative real value radius r_a such that: $\forall\, c \in C\ \forall\, v \in V : c \in A(v) \Leftrightarrow \sqrt{(x_c - x_v)^2} \leq r_c + r_v$

We can easily think of it as: the point x_a represents ideal position in the 1-dimensional space of opinions. The radius can be seen as candidate's charisma or voter's willingness to compromise.

Candidate Range

We say that P satisfies CR if candidates in C can be ordered so that each of the sets v_i forms an interval of that ordering. CR is special case of VCR, where we require all the voters to have radii equal to zero

Voter Range

We say that P satisfies VR if the voters in V can be reordered so that for every candidate c_i the voters that approve c form an interval of that ordering. VR is a special case of VCR, where we require all the candidates to have radii equal to zero

Voter Interval and Candidate Interval

Faliszewski (2021) showed that CR and VR are equivalent to respectively VI and CI introduced by Elkind and Lackner (2015).

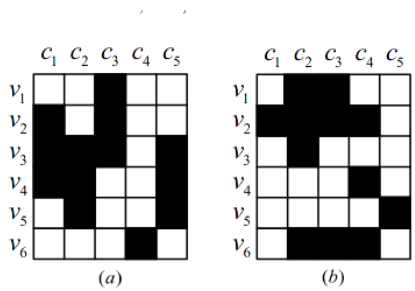

Figure 3.1 CR and VR Temporary image

Larger Domain

(WIP: Zmienić na mój przykład i opisać dlaczego nie vr i nie cr)

The VCR model has larger domain than the VR and CR combined. For example, election with candidate set $C = \{a,b,c,d\}$ and voters with approval sets $\{a,b\}$, $\{b,c\}$, $\{b,d\}$, and $\{a,b,c,d\}$ is VCR, but neither VR nor CR.

Chapter 4

Voter Candidate Range Profile Detection

In this chapter we present our algorithms for deciding if a given approval profile has the VCR property.

4.1 Profile Structure Detection

To take advantage of the restrictions defined in Section 3, we need to design detection algorithms. They can be used to check whether given election profiles satisfy particular structural properties. For example, to check the VR (CR) property it is enough to check for the *consecutive ones property* in the preference matrix.

Definition 1. *A binary matrix (i.e., a matrix consisting of zeros and ones) has the consecutive ones property (C1P) if its columns be permuted in such a way that for each row the ones are consecutive, i.e., the 1s form an interval.*

In this work we refer to the term *C1P* more broadly, using it interchangeably, for an analogous scenario where we permute the rows and look for the consecutive 1s in every column in a given matrix. Booth and Lueker (1976) provided a linear-time C1P detection algorithm based on the PQ-Tree data structure.

Proposition 1 (Elkind and Lackner (2015))**.** *Deciding whether an approval profile satisfies VR or CR is possible in $O(m \cdot n)$ time.*

Proof. Let $C = \{c_1, ..., c_m\}$ and $V = \{v_1, ..., v_n\}$, we create a $n \times m$ matrix A (a_{ij}): for each $i \in [n]$ and $j \in [m]$: $a_{ij} = 1$ if $c_j \in A(v_i)$ and 0 otherwise. We can see that for the VR property, permuting the columns of matrix A so that ones form an interval in each row is equivalent to permuting candidates so that the set of candidates approved by each voter forms an interval. For the CR case, we just need to transpose the matrix created for the VR. □

Our focus lies on the one dimensional VCR property. Having preference matrix as input we have to provide for each agent (voter/candidate) a point on the real line and a nonnegative real value representing radius, so that these parameters satisfy the VCR property.

4.2 Integer Linear Program

An algorithm based on ILP solving is an excellent choice for the first attempt at designing a detection algorithm. It provides an efficient and relatively simple solution. Furthermore, it can be used as a fast benchmark for validating future solutions. Below we provide an ILP for the VCR profile recognition, together with additional preliminary steps.

Transforming the Definition of VCR into Linear Constraints

Before formulating an ILP for detecting VCR profiles, we have to redefine the definition of VCR in order to make it appropriate for an ILP solver. The definition is based on the euclidean distance which cannot be directly represented in an ILP. Equation 4.2 shows transformation of the VCR definition into linear inequalities.

(WIP: Czy powtarzam *VCR definition* za często?)

As a short reminder from Section 3.2, for the candidates C and voters V, 1D-VCR is defined as:

$$\forall\, c \in C\, \forall\, v \in V : c \in A(v) \Leftrightarrow |x_c - x_v| \leq r_c + r_v \tag{4.1}$$

Where x_a and r_a represent agent's point on the real line and their radius respectively.

(WIP: W jaki sposób mogę zwięźle, bez ponownego wpropadzania wszystkich oznaczeń, przypomieć definicję? O ile w ogóle jest to potrzebne...)

$$\begin{aligned} &|x_c - x_v| \leq r_c + r_v \\ &\iff -(r_c + r_v) \leq x_c - x_v \leq r_c + r_v \\ &\iff x_c - x_v \leq r_c + r_v \;\; \vee \;\; -(x_c - x_v) \leq r_c + r_v \end{aligned} \tag{4.2}$$

(WIP: Poprawić formatowanie, jak?)

The constraint obtained from the equation 4.2 is not suitable yet for an ILP because it contains logical *OR* condition which again cannot be directly represented in an ILP. To transform it into linear form, we apply commonly called *BigM* technique, as shown below.

$$
\begin{aligned}
x_c - x_v &\leq r_c + r_v + M(1 - z_1) \\
-(x_c - x_v) &\leq r_c + r_v + M(1 - z_2) \\
z_1 + z_2 &\geq 1 \\
z_1, z_2 &\in \{0, 1\}
\end{aligned}
\tag{4.3}
$$

First, we have to introduce a large constant M (larger than $max(|x|)$) and two binary variables z_1 and z_2. Now if any of the z_i is 0 then the corresponding inequality will be true, and the additional constraint $(z_1 + z_2 \geq 1)$ guarantees that at least one of the z_i will be 1.

(WIP: Trochę namieszane wytłumaczenie, bo tak naprawdę problem z *OR* jest tylko dla wariantu if $c \notin A(v)$, czyli $x_c - x_v + M(1 - z) \geq r_c + r_v + 1$ ...)

The BigM technique is common enough, for most solvers to provide additional built-in mechanisms for applying it. Using them is usually recommended, as it allows a solver for better internal optimization. In Gurobi solver we use *indicator constraint*, which allows one to create versatile constraints in form of: $z = f \rightarrow a^T x \leq b$. The constraint states that if the binary indicator variable z is equal to f, where $f \in \{0, 1\}$, then the linear constraint $a^T x \leq b$, where a and b are data coefficients vectors and x represents the decision variables, should hold. On the other hand, if $z = 1 - f$, the linear constraint may be violated.

(WIP: BigM jest używane do reprezentacji różnych 'funkcji', nie tylko *OR*, np. *max*, *abs*, etc., czy też o tym wspomnieć?)

Detection Algorithm

Final formulation of the ILP for recognising VCR property in a given election profile is shown in Figure 4.4. The program accepts preference matrix A as input, and if solution exists it outputs every agent's position x_a and radius r_a. Additionally, we use two binary variables p and q, required by the indicator constraints.

check feasibility :

subject to :

$$
\begin{aligned}
& x_a \in \mathbb{R} && \forall \text{ a} \in C \cup V \\
& r_a \in \mathbb{R}^{0+} && \forall \text{ a} \in C \cup V \\
& p_{cv} \in \{0,1\} && \forall \text{ c} \in \text{C} \; \forall \text{ v} \in V \\
& q_{cv} \in \{0,1\} && \forall \text{ c} \in \text{C} \; \forall \text{ v} \in V \\
\text{if } c \in A(v) : & && \\
& x_c - x_v \leq r_c + r_v && \\
& -(x_c - x_v) \leq r_c + r_v && \\
\text{if } c \notin A(v) : & && \\
& p_{cv} == 1 \rightarrow x_c - x_v \geq r_c + r_v + 1 && \\
& q_{cv} == 1 \rightarrow -(x_c - x_v) \geq r_c + r_v + 1 && \\
& p_{cv} + q_{cv} >= 1 &&
\end{aligned}
\tag{4.4}
$$

4.3 (WIP) My VCR Detection Algorithm

4.4 (WIP) C1P Detection - ILP vs PQTree Performance Comparison

Chapter 5

Voter Candidate Range Profiles

In this section we explore VCR profiles domain and describe its found properties (WIP: properties == Cechy/zalezności statystyczne). The main goal is to compare size of the domain with other well known domains such as VR and CR.

5.1 Domain analysis

We've performed several experiments. (WIP: tense: present perfect vs past simple vs present?) In each of them we check how many profiles have VCR property but not VR nor CR. For this we use the detection algorithm described in the section 4.2. For the first one we use every combination of the profiles for the given election size as the input. (WIP: czy to na pewno kombinacje i czy nie jest to zbyt pomieszane?) For the second experiment we generate random VCR profiles.

All the resources and implementation details related to the task are included in the appendix **??**.

(WIP: czy w ogóle pisać o szczegółach implementacyjnych)

(WIP: Jak często rozbijać tekst na akapity?)

5.2 (WIP) How many VCR profiles are not VR nor CR?

Experiment 1 - small profiles

For the small profiles we performed a complete search of the domain. We checked the elections with the following sizes (M candidates N voters): $[(3,3),(4,4),(5,5),(4,6),(6,4)]$. The results as seen in the table 5.1 show that VCR domain is relatively large, however only limited number of profiles are not VR nor CR.

	3C 3V	4C 4V	5C 5V	4C 6V	6C 4V
Count	2^9	2^{16}	2^{25}	2^{24}	2^{24}
VCR	506	57832	19846082	10893736	10893736
VR	-	-	-	-	-
CR	-	-	-	-	-
VCR !COP	0	96	501000	144480	144480

Table 5.1 Temporary table - Small profiles NCOP/VCR

WIP : More advanced conclusions?

WIP : Prettify the table

WIP : Make some charts, write statistics, show examples

Profile examples

Example of 5C 5V election that is VCR but not VR nor CR.

WIP: Replace with a drawing and explain why not VR/CR

```
Approval Set
    [1., 1., 0., 0., 1.]
    [1., 1., 1., 1., 1.]
    [1., 0., 0., 0., 0.]
    [0., 0., 0., 1., 1.]
    [0., 0., 1., 0., 1.]

Candidates:
    Candidate(id='A', x=3.0, r=1.0),
    Candidate(id='B', x=2.0, r=0.0),
    Candidate(id='C', x=0.0, r=0.0),
    Candidate(id='D', x=1.0, r=0.0),
    Candidate(id='E', x=1.0, r=1.0)

Voters:
    Voter(id='V0', x=2.0, r=0.0),
    Voter(id='V1', x=0.0, r=3.0),
    Voter(id='V2', x=3.0, r=0.0),
    Voter(id='V3', x=1.0, r=0.0),
    Voter(id='V4', x=0.0, r=0.0)
```

Experiment 2 — large profiles

For the larger size of the profiles we generated a suitable number of random VCR profiles and again checked how many of them are not VR nor CR. To create random VCR profile we generated points x_a and radii r_a for every agent $a \in C \cup V$ and then created preference matrix based on the VCR definition.

Additionally, to have a broader view of the domain the experiment is split into sub-experiments based on the different random distributions (WIP: Czy distributions jest ok?) used to generate profiles.

Uniform distribution

Uniform distribution generation parameters:

1. Candidate : rMin=0, rMax=40, xMin=0, xMax=100
2. Voter : rMin=0, rMax=20, xMin=0, xMax=100

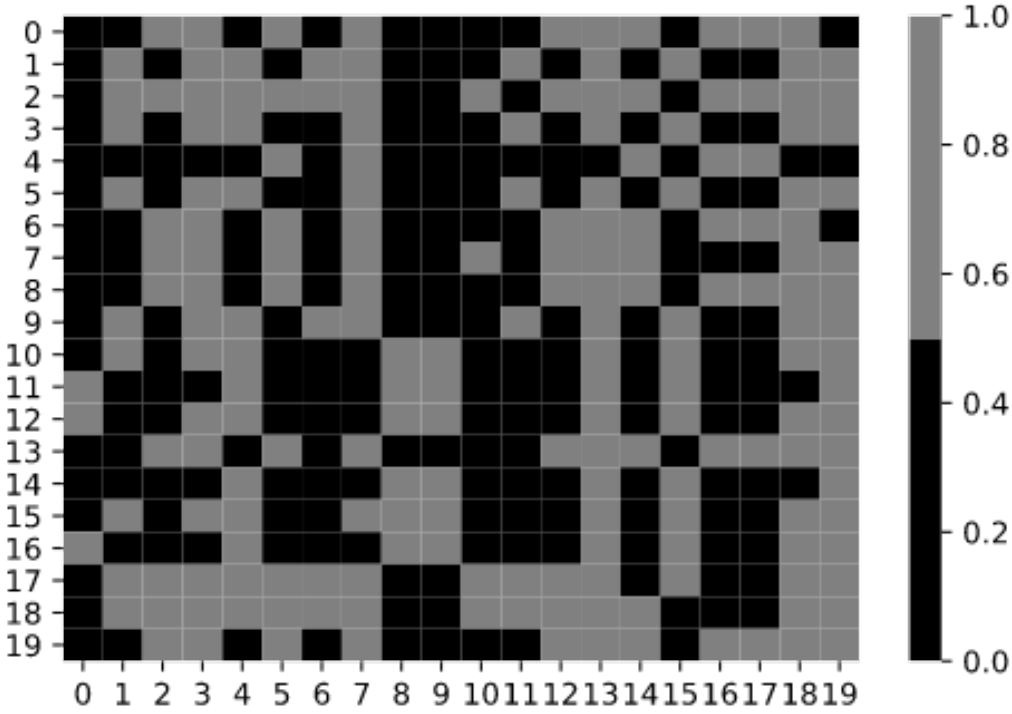

Figure 5.1 Temporary Image - Uniform NCOP HeatMap

Double Overlapping Gauss

Double Gauss distribution generation parameters:

1. C=20
2. V=20
3. proportionC=0.7, proportionV=0.7,
4. meanMajor=-1.5, meanMinor=1.5, std=0.8,
5. radiusConst=0.7

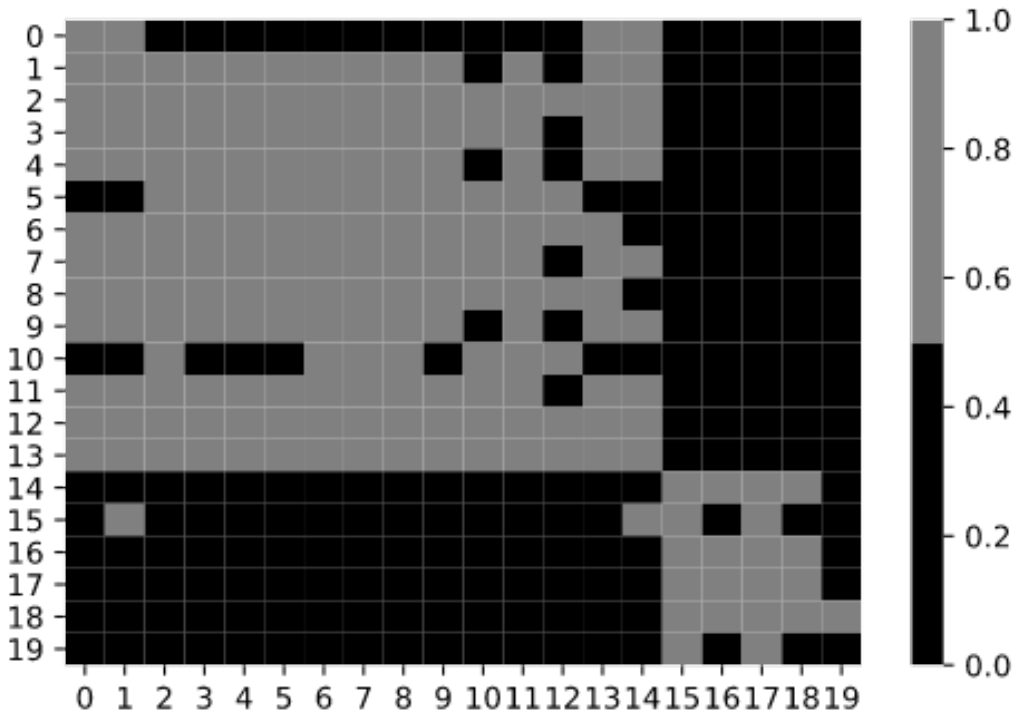

Figure 5.2 Temporary Image - Double Gauss HeatMap

Chapter 6

HD-Minimax Scoring Rule Algorithms

6.1 Hamming Distance Minimax Approval Voting

Hamming Distance

Given two sets X, Y their Hamming distance is $d_{Ham}(X,Y) = |X \setminus Y| + |Y \setminus X|$. Alternatively for a vector representation we define the Hamming distance between vectors of the same size a, b as $d_{Ham}(a,b) = |a \oplus b|$.

Minimax Approval Voting (MAV)

Scoring rule introduced by Brams et al. (2007). Selects a committees W that minimizes the maximum Hamming distance between W and voters' approval ballots, i.e. *MAV mininimizes* $max_{v \in V}\ d_{Ham}(A(v), W)$. Intuitively MAV follows the egalitarian principle (WIP: Co to dokładnie / czy istnieje definicja?), it selects committees that minimize the dissatisfaction of the least happy voter.

(WIP: poprawić zdanie)

(WIP) Computational complexity

Computing the winning committee for MAV is known to be NP-hard, see **?** (WIP: cytacje). Nevertheless, Liu and Guo (2016) provide polynomial-time algorithms for computing MAV results, for the special case of the elections with candidate range or voter range properties.

(WIP: Poprawić zdanie)

6.2 (WIP) My algorithm for VCR profiles

(WIP) Current approches

Greedy

1. Sort by ?
2. ?

Dynamic

1. Recursive
2. branch and bound
3. ?

Augumented Data Structure

1. ?

Chapter 7

(WIP) Approval deletion for VCR profile to become VR or CR

Chapter 8

Control Problem

8.1 Overview

8.1.1 AV-CC-DV for Single Peaked (VR = CI = row C1P = $r_c = 0$)

Algorithm from Faliszewski et al. (2009)

1. kandydat p na osi, pozostali na lewo i prawo
2. wybór przeciwników (groźny kandydat):
   (a) pierwszy na lewo c_L i pierwszy na prawo c_R takich, że $score(c_?) \geq score(p)$
   (b) powtarzamy dla kolejnych lewych c_{L_i} takich, że $score(c_{L_i}) > score(c_{L_{i-1}})$
   (c) analogicznie dla prawych c_{R_i} takich, że $score(c_{R_i}) > score(c_{R_{i-1}})$
3. usuwanie głosów - zaczynając od najbliższego prawego przeciwnika c_R (dla lewych - odbicie lustrzane)
   (a) notacja v: $b(v)$ początek przedziału (lewy) i $e(v)$ koniec przedziały (prawy)
   (b) wybieramy głosujących $\widehat{V}$ takich że $\{v \in V | \, b(v) \in (p, c_R]\}$
   (c) sortujemy $\widehat{V}$ malejąco po $e(v)$ (od najbardziej prawego końca przedziału)
   (d) usuwamy $scoreDelta(p, c_R) + 1$ głosów
4. powtarzamy do skutu, przeliczając aprobaty co iterację

8.1.2 AV-CC-DV for Candidate Range = VI = column C1P = $r_v = 0$

Algorithm from Magiera and Faliszewski (2014)

1. notacja CR dla c: $v_{f(c)}$ początek przedziału (lewy/górny) i $v_{e(c)}$ koniec przedziału (prawy/dolny)
2. kandydat p
3. Głosujący - wybieramy głosujących $V' = \{v \in V | \, p \notin A(v)\}$
4. Przeciwnicy - wybieramy kandydatów $\widehat{C} = \{c \in C \setminus \{p\} | \, scoreDelta(c, p) \geq 0\}$
5. Usuwanie głosów
   (a) zaczynając od przeciwnika c_N takiego, że $v_{e(c_N)}$ jest minimalne
   (b) usuwamy $deltaScore(c_N, p) + 1$ głosów, zaczynając od końca przedziału $v_{e(c_N)}$
6. powtarzamy do skutu, przeliczając V' i $\widehat{C}$ co iterację

8.2 AV-CC-DV for the VCR profiles

Notation used in the algorithm and later in the proofs:
x_a, r_a — point and radius
$b(a) = x_a - r_a$ — range beginning (left)
$e(a) = x_a + r_a$ — range end (right)
p — candidate chosen to be a winner
$\widehat{C} = \{c \in C \setminus \{p\} | \, scoreDelta(c, p) \geq 0\}$ — opponents (dangerous candidates)
$V' = \{v \in V | \, p \notin A(v)\}$ — potential voters to delete

We use “smart” greedy strategy for voter deletion:

1. We pick a leftmost opponent c_L that is the one with the minimal $b(c_L)$
2. We consider only voters who approve of c_L that is, $\widehat{V} = \{v \in V' | \, c_L \in A(v)\}$
3. We need to delete $deltaScore(c_L, p) + 1$ rightmost voters from $\widehat{V}$ that is the ones with the maximal $e(v)$, if it’s not possible we fail
4. We repeat the above steps until we succeed, recalculating $\widehat{C}$ and $\widehat{V}$ after each iteration

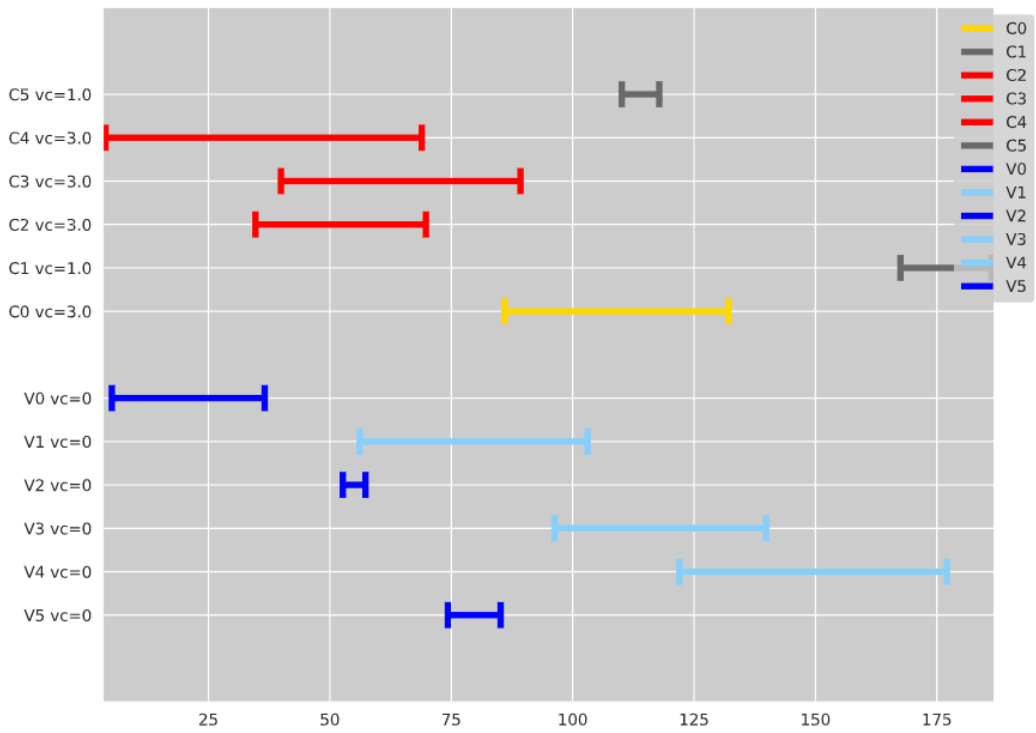

Figure 8.1 Temporary Image - VCR !C1P Election
yellow — candidate p
red — opponents
lightblue — voters whitelist
darkblue — possible voters to delete
grey — agents that can be ignored

8.2.1 Sketch Proof - CR case

We will show how the VCR algorithm is equivalent to the CR one from Magiera and Faliszewski (2014). Having voters in CR order, that is, they are still represented by a point x_v on the $\mathbb{R}$ line but have radius 0. Example in Figure 8.2.

Choosing opponents

In the CR algorithm, we choose candidates to process, ordered by $v_{e(c)}$ ascending. We can see that it is equivalent to picking the leftmost $(min(b(c)))$ candidate from the set of opponents $\widehat{C}$ in the VCR algorithm, because it will also have the minimal $v_{e(c)}$.

There is a special case of "contained candidate", for which the above statement is not true. In Section 8.2.3, we explain in detail why it is irrelevant for the overall correctness of the algorithm.

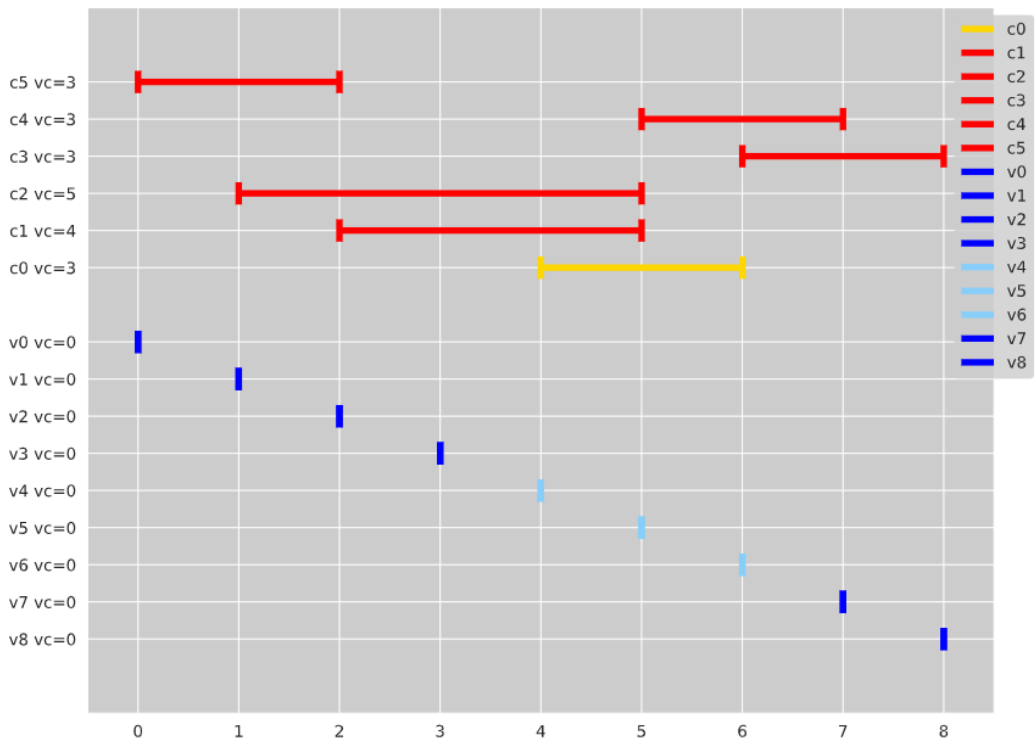

Figure 8.2 Temporary Image - CR Election

Voter deletion

Now, once we now how to pick an opponent c_L, we will show that choosing voters is equivalent as well. Notion of the beginning/end of the interval in the CR order is equivalent to the left/right end of the range in the VCR setting. Both algorithms try to delete $scoreDelta(p, c_L) + 1$ voters, CR one starts deleting voters from the end of interval, while VCR does virtually the same by deleting the rightmost voter first ($max(e(v))$).

(WIP: Czy rozumowanie *lewo prawo* na osi $\mathbb{R}$ dla VCR, jest analogiczne do *początek koniec* interwału dla CR, wystarczy jako dowód?)

8.2.2 Sketch Proof - VR case

We will show how the VCR algorithm is equivalent to the VR one from Faliszewski et al. (2009). Having candidates in VR order, that is, they are still represented by a point x_c on the $\mathbb{R}$ line but have radius 0. Example in Figure 8.3.

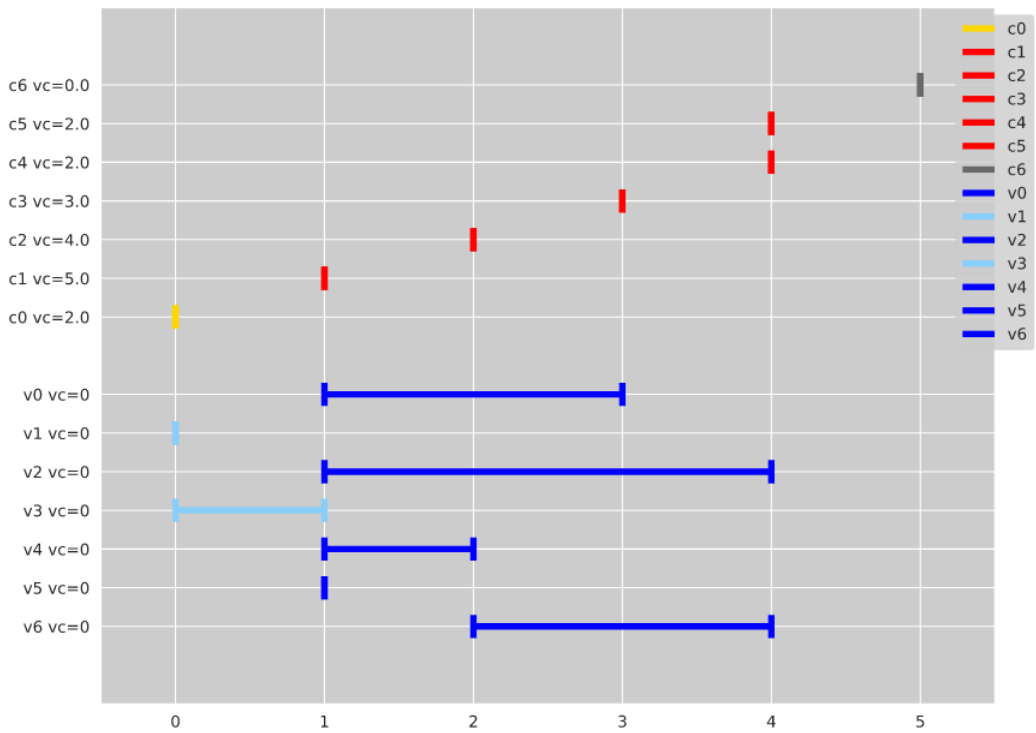

Figure 8.3 Temporary Image - VR Election

Choosing opponents

VR algorithm picks dangerous candidates to the left and right of candidate p, however later it proceeds only with defeating the right ones. Once finished with them, it does the "mirror flip" of the universe to process candidates on the left side. Even though the VCR algorithm picks leftmost candidate, we will show it is equivalent to the VR strategy under some basic assumptions.

Following example in Figure 8.3, let's assume that candidate p always will be the first candidate in the VR order. Then we can see that the VCR algorithm strategy of choosing leftmost candidate will be the same as the VR algorithm choosing first on the right candidate.

This assumption is correct because, having candidates in order $[c_L, p, c_R]$, if there was a voter v who approved of c_L and c_R it would also have to approve of p (because of VR property), so we could not delete v anyway.

Voter deletion

Both algorithms have the same voter deletion strategy.

8.2.3 Sketch Proof - VCR (not C1P) case

Now we will show why our solution is correct in the general VCR setting, even when the profile does not have VR/CR property, example in Figure 8.1. The proof is split into two stages, similarly to the previous ones. But first we will explain one special case of the election, that is worth examining.

Special case - contained candidates

In the VCR model it is possible that some candidates have their interval fully contained in the other's interval. Every voter who would approve the smaller candidate would also approve the larger one, however not vice versa. For this case, our strategy of choosing voters to delete might seem suboptimal, at first glance. In the minimal viable example from Figure 8.4, the algorithm would delete v_1 first, however, to defeat c_0, we need to delete 2 voters, so v_0 will be deleted as well.

The example illustrates why we do not have to worry about this special case. Every contained voter will have as much or less total votes than the larger candidate, he/she might not even be a dangerous candidate at all. Now we see that in the process of defeating the larger candidate we will automatically defeat the smaller one.

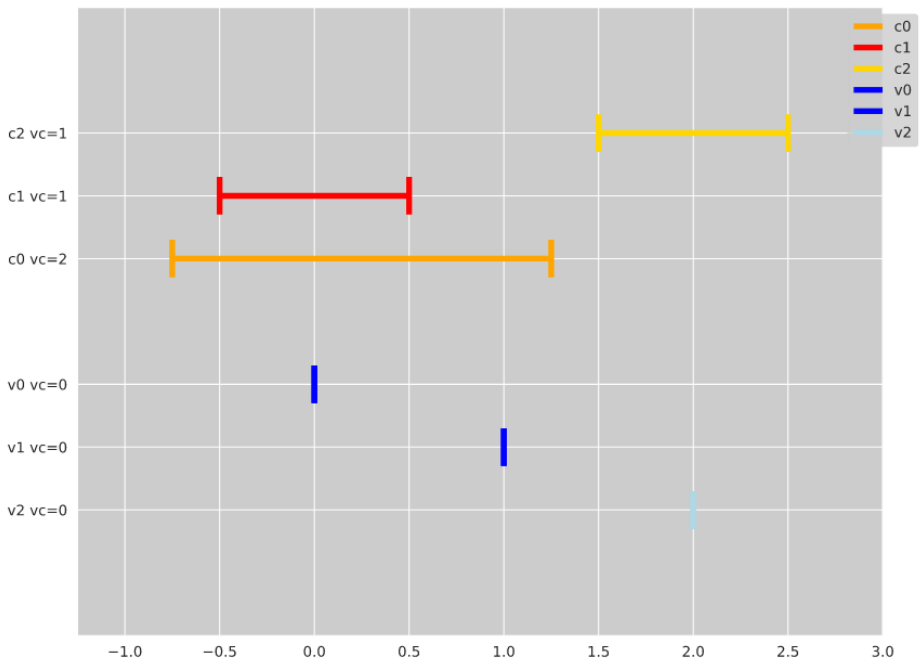

Figure 8.4 Temporary Image - Special Case

Choosing opponents

We consider only dangerous candidates (opponents), that is the ones with score higher or equal to the score of p. We process opponents one by one, starting with the one who has the leftmost beginning of the range, i.e., $min(b(c))$. After each iteration, we recompute the set of dangerous candidates. Thanks to this we are sure that all the steps we perform are necessary to find a solution (if it exists), hence optimal.

Voter deletion

Having currently processed opponent c_L, we consider only the voters who approve of him/her and do not approve of p, that is the voters whose range beginning is inside the candidate's range $\{v \in V' \mid b(v) \in [b(c_L), e(c_L)]\}$. We need to delete $scoreDelta(c_L, p) + 1$ voters to defeat the opponent. By picking the voters in order of the rightmost end of the range, i.e., $v_R = max(e(v))$, we try to maximize the number of other candidates who might be in the range $[b(v_R), e(v_R)]$. We don't have to worry about any candidate to the left, because of our leftmost candidate picking strategy, they are already defeated.

Chapter 9

Evaluation

9.1 Conclusions

9.2 Future Work

Bibliography

Booth, K. S. and Lueker, G. S. (1976). Testing for the consecutive ones property, interval graphs, and graph planarity using pq-tree algorithms. *Journal of Computer and System Sciences*, 13(3):335 – 379.

Brams, S., Kilgour, D., and Sanver, M. R. (2007). *A Minimax Procedure for Negotiating Multilateral Treaties*, pages 265–282.

Elkind, E. and Lackner, M. (2015). Structure in dichotomous preferences. *CoRR*, abs/1505.00341.

Faliszewski, P. (2021). An analysis of approval-based committee rules for 2d-euclidean elections.

Faliszewski, P., Hemaspaandra, E., Hemaspaandra, L. A., and Rothe, J. (2009). The shield that never was: Societies with single-peaked preferences are more open to manipulation and control. *CoRR*, abs/0909.3257.

Faliszewski, P., Skowron, P., Slink, A., and Talmon, N. (2017). *Multiwinner Voting: A New Challenge for Social Choice Theory*. AI Access.

Liu, H. and Guo, J. (2016). *Parameterized Complexity of Winner Determination in Minimax Committee Elections*, page 341–349. AAMAS '16. International Foundation for Autonomous Agents and Multiagent Systems.

Magiera, K. and Faliszewski, P. (2014). How hard is control in single-crossing elections? *Frontiers in Artificial Intelligence and Applications*, 263:579–584.